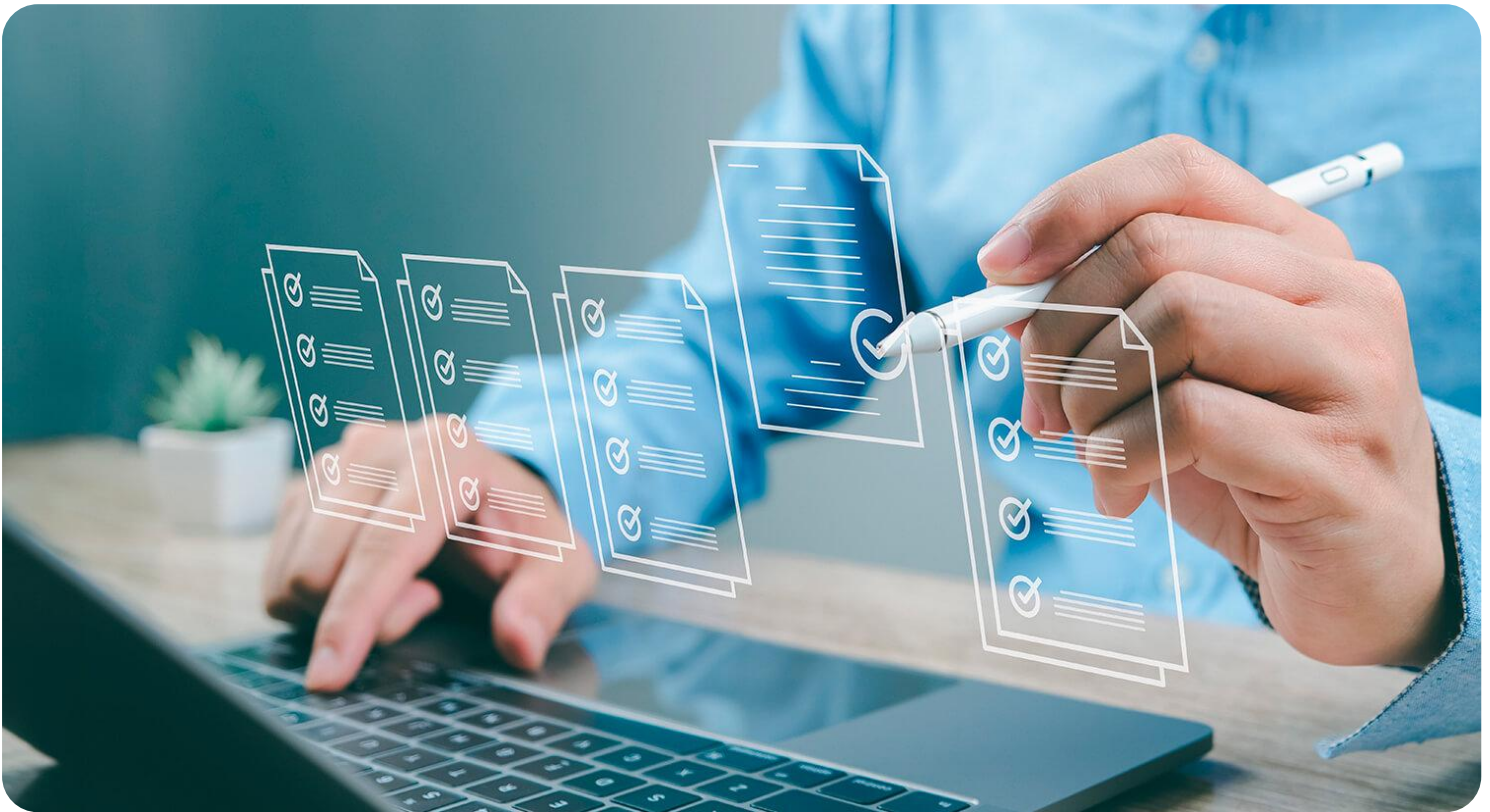




Semana 8

Desarrollo de Aplicaciones Móviles (DSY2204)

Formato de respuesta

Nombre estudiante: Rodolfo Ulloa	
Asignatura: Desarrollo de Aplicaciones Móviles	Carrera: Ingeniería en Desarrollo de Software
Profesor: Enrique Urra	Fecha: 03/10/2026

Descripción de la actividad

En esta octava semana, se realizará la actividad sumativa 3, llamada " Preparación de una aplicación móvil Android/Kotlin para su distribución". Deberán integrar a la app creada en el framework Android Studio y Kotlin el front end y back end, y preparar la aplicación móvil para su publicación en una distribución de descarga de app. Entrega de encargo con documentación del desarrollo y código del proyecto comprimido en .zip.

Instrucciones específicas

A continuación, se detallan las instrucciones y requisitos de la actividad sumativa 2, deberás leer cada uno de ellos y apuntar al cumplimiento.

Características del producto:

- Integrar en la aplicación móvil de accesibilidad para personas con discapacidad sensorial auditiva desarrollada en la experiencia de aprendizaje 1, el front end y back end en Kotlin, además preparar y publicar el APK firmado.
- La aplicación móvil deberá ser diseñada con componentes UI específicamente para personas con discapacidad sensorial auditiva, facilitando a los usuarios su comunicación (escribir y hablar) en su entorno cotidiano, mejorando su calidad de vida al transformar sus dispositivos móviles en una herramienta que los integra al mundo real y digital.

Requerimientos generales:

Para el desarrollo de la entrega solicitada considerar los siguientes aspectos:

- El desarrollo de la entrega solicitada, las y los estudiantes deben considerar los siguientes aspectos:
- Integrar el front end (las views Login, Registro, Recuperar contraseña, HomeMenú, Escribir, Hablar y BuscarDispositivo, usando geolocalización, fragments, viewgroups, content providers, widgets, palette texts, buttons y palette widgets) y el back end (base de datos Google Firebase para la permanencia de los datos, reglas de conexión con los objetos y la configuración de parámetros para la autenticación de usuarios registrados) en Kotlin a la app móvil desarrollada en las experiencias de aprendizaje 1 y 2.
- Deberá implementar servicios para el manejo de los datos, como la conexión a la base de datos, autorización y autenticación de usuarios, CRUD para la gestión de usuarios y para las funciones de las views Escribir, Hablar y BuscarDispositivo.
- Testear con pruebas unitarias (en JUnit, Mockito, Robolectric, Espresso, TDD y Firebase Test Lab) la app móvil desarrollada, para luego generar el APK, firmarlo y publicarlo en una plataforma de descarga (gratuita), simulando la tienda Google Play Store.

Requerimientos técnicos

- Desarrollar la aplicación con el Framework Android Studio.
- Generar y compartir el repositorio Git del proyecto.
- Integrar el lenguaje de programación Kotlin al código fuente de la app móvil.
- Integrar los componentes UI de Android Studio y Kotlin.
- Integrar Firebase para gestionar la persistencia de los datos y la autorización a ellos.
- Desarrollar pruebas unitarias en Kotlin.
- Generar el APK.
- Firmar y publicar el APK de la aplicación móvil desarrollada.

Entrega de encargo con documentación

Esta actividad tendrá dos partes y se entregará todo en **un solo ZIP**.

En la primera parte, tras finalizar el desarrollo del proyecto desde Android Studio, se deberá comprimir la carpeta del código del proyecto completo en formato .zip desde Git, además de generar, firmar y publicar el APK de la app y adjuntarlo en espacio indicado en el AVA.

En la segunda parte, se deberá documentar todo el proceso de desarrollo del proyecto, describiendo el rendimiento de las views diseñadas y de las pruebas unitarias, las restricciones y describir el seguimiento de los 5 riesgos técnicos del proyecto definidos en la experiencia de aprendizaje 1, además de describir los resultados de los accionables de mitigación de cada riesgo definido. Todo ello **en este mismo documento de formato de respuesta, a continuación de estas instrucciones** y subirlo en el ZIP que entregarás con el desarrollo del Proyecto y este documento del formato de respuesta debe subirse en formato PDF.

- Entregar el producto totalmente finalizado, según el avance solicitado en esta entrega.
- No se aceptarán entregas fuera de plazo ni por otro medio que no sea AVA.
- Todos los productos desarrollados con IA serán evaluados con nota mínima.

Restricciones en esta entrega

La aplicación móvil debe ser adaptativa a múltiples dispositivos Android que cuenten con una conexión activa a internet.

Formato del texto:

- Fuente: Arial.
- Tamaño de fuente: 12 puntos.
- Interlineado: 1,5 líneas.
- Alineación del texto: a la izquierda.

Iniciar documentación desde la siguiente página.

Datos del documento

Histórico de revisiones

Versión	Fecha	Descripción/cambio	Autor
1.0	03/10/2026	Versión inicial: integración front end y back end con Firebase, pruebas, firma y publicación del APK	Rodolfo Ulloa

Información del proyecto

Organización	Duoc UC
Sección	DSY2204
Proyecto (nombre)	AccesiPlus
Fecha de inicio	21/08/2026
Fecha de término	03/10/2026
Patrocinador principal	Duoc UC
Docente	Enrique Urra

Alcance del proyecto.

Problema, solución propuesta y contexto del proyecto.

Problema

Las personas con discapacidad sensorial auditiva enfrentan barreras de comunicación en situaciones cotidianas, como la atención en salud, el comercio, el transporte o el trabajo, donde la interacción depende de la voz. En las entregas anteriores, AccesiPlus resolvía parte del problema con texto a voz, pero sus datos vivían en memoria: las cuentas y las frases se perdían al cerrar la aplicación y no existía una versión instalable fuera de Android Studio.

Solución propuesta

AccesiPlus 1.0.0 es una aplicación móvil nativa para Android que integra un front end accesible (texto grande, alto contraste y botones de 56 dp) con un back end en Google Firebase. Permite comunicarse en dos sentidos:

- **Escribir:** convierte en texto grande lo que dice otra persona (voz a texto) o lo que el usuario teclea.
- **Hablar:** el teléfono dice en voz alta lo que el usuario escribe o elige entre frases rápidas por categoría.
- **BuscarDispositivo:** registra audífonos, implantes y teléfonos con la ubicación donde quedaron, para encontrarlos en el mapa.

Cada usuario tiene su cuenta (Firebase Authentication) y sus datos se guardan de forma persistente y privada en Firebase Realtime Database. La aplicación se entrega como un APK firmado con un keystore propio y publicado en Firebase App Distribution, simulando la distribución por Google Play Store.

Contexto tecnológico

- Lenguaje y UI: Kotlin 2.2, Jetpack Compose (BOM 2026.02.01) y Material Design 3; Android Gradle Plugin 9.3.
- Navegación: Navigation Compose 2.10.2. Se usó la versión estable vigente para mantenerla alineada con el resto de las dependencias AndroidX actualizadas en

esta entrega (lifecycle 2.11, activity 1.13, Compose BOM 2026). La API utilizada (NavHost, composable, navigation, popUpTo y argumentos de ruta) es la misma de las versiones 2.5.x y 2.8.x trabajadas en el curso.

- Compatibilidad: minSdk 24 (Android 7.0) y targetSdk 37; paquete `cl.duoc.rulloa.accesiplus`.
- Arquitectura por capas: UI (Compose) → ViewModel (StateFlow y viewModelScope) → Repository (interfaces con corrutinas, `await()` y `callbackFlow`) → Firebase y Room.
- Back end: Firebase Authentication (correo y contraseña) y Realtime Database con persistencia sin conexión y reglas de seguridad por usuario.
- Componentes Android: geolocalización (FusedLocationProviderClient), fragment con ViewGroup clásico, ContentProvider respaldado por Room, App Widget y Palette.
- Extensiones KTX: `core-ktx`, `lifecycle-viewmodel-ktx`, extensiones Kotlin de Firebase, `kotlinx-coroutines-play-services`, `room-ktx`, `palette-ktx` y `fragment-ktx`.

Estructura de datos y seguridad

Todos los datos cuelgan del identificador del usuario autenticado; no hay nodos compartidos:

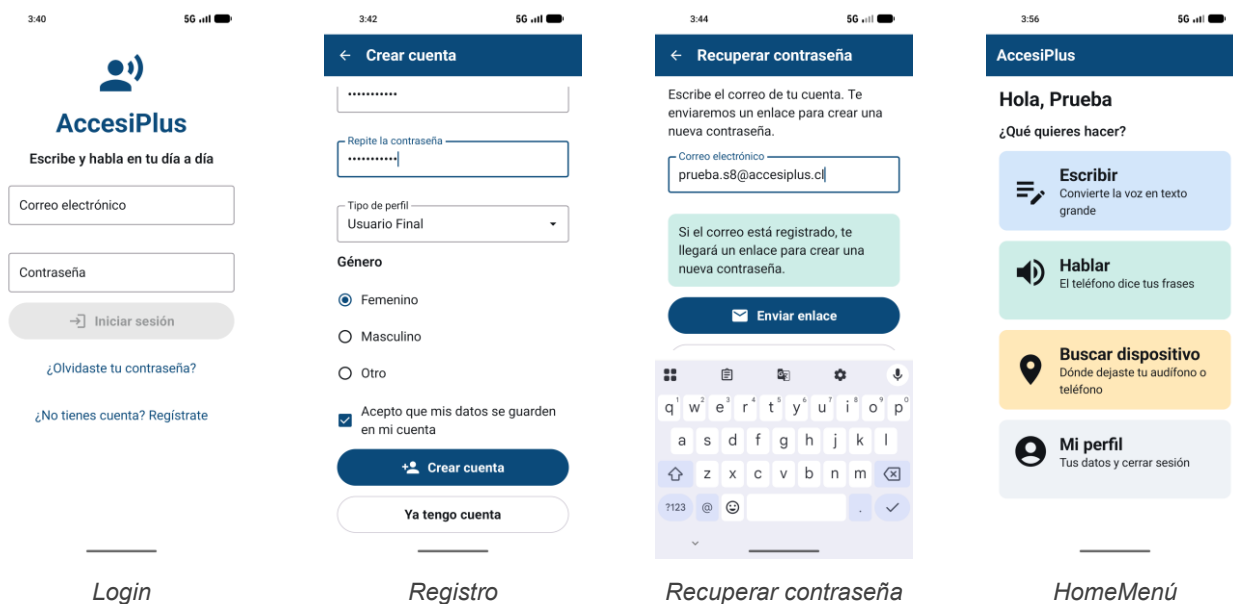
- `users/{uid}/profile`: nombre, correo, tipo de perfil, género y fecha de creación.
- `users/{uid}/phrases/{id}`: texto, categoría, favorita, cantidad de usos y fechas.
- `users/{uid}/devices/{id}`: nombre, tipo, notas, latitud, longitud y fecha de la ubicación.

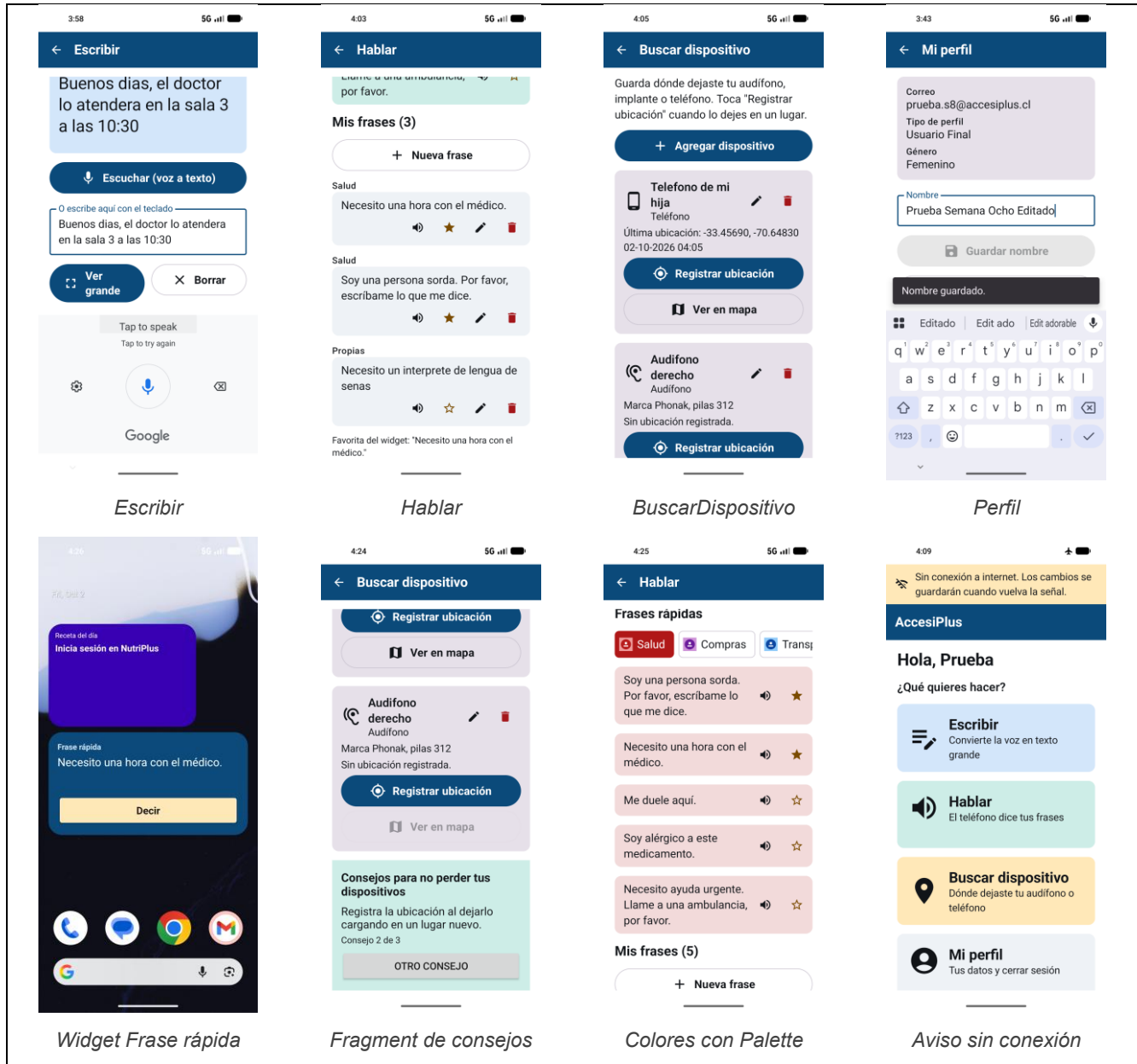
Las reglas publicadas en la consola de Firebase niegan todo acceso por defecto y solo permiten leer y escribir `users/$uid` cuando `auth.uid === $uid`. Además validan los campos, sus tipos, los largos máximos y los rangos de latitud y longitud. Las frases rápidas por categoría están en el código, por lo que no se comparte ningún nodo entre usuarios.

Funciones esperadas del software a desarrollar.

Las funciones implementadas se muestran con capturas de la aplicación en ejecución (emulador Pixel 5, Android 17), que sirven como mockup de cada view:

- **Autenticación real:** login, registro y recuperación de contraseña con Firebase Authentication; los errores de Firebase se traducen a español simple.
- **Sesión persistente y navegación protegida:** sin sesión solo se accede a Login, Registro y Recuperar contraseña.
- **Perfil:** ver datos, editar el nombre, cerrar sesión y eliminar la cuenta junto con sus datos.
- **CRUD de frases y dispositivos:** crear, leer, editar y eliminar en Firebase desde Escribir, Hablar y BuscarDispositivo.
- **Geolocalización:** registro de la ubicación con permiso en tiempo de ejecución y apertura en la app de mapas.
- **Funcionamiento sin conexión:** aviso visible y guardado local que se sincroniza al volver la red.
- **Widget, ContentProvider, Fragment y Palette:** acceso rápido a la frase favorita desde la pantalla de inicio y colores por categoría.





Rendimiento esperado del software a desarrollar.

Descripción de cada view según su propósito, componentes UI, flujo de navegación y manejo de la entrada.

View de Login (LoginScreen.kt)

- **Propósito:** autenticar al usuario con su correo y contraseña en Firebase Authentication.
- **Componentes UI:** ícono y título, dos OutlinedTextField (correo con teclado de email y contraseña oculta), botón principal grande y dos TextButton hacia Registro y Recuperar contraseña.
- **Flujo de navegación:** con login correcto, la guardia de sesión reemplaza toda la pila de navegación por el HomeMenú, por lo que el botón «Atrás» no vuelve al Login.
- **Manejo de entrada:** valida el formato del correo mientras se escribe; el botón se habilita solo con datos completos; la tecla Enter inicia sesión; los errores se anuncian a TalkBack (liveRegion).

View de Registro (RegisterScreen.kt)

- **Propósito:** crear la cuenta en Firebase Authentication y el perfil en users/{uid}/profile.
- **Componentes UI:** cuatro campos de texto, combo box de tipo de perfil (ExposedDropDownMenuBox), radio buttons de género, checkbox de aceptación y botón grande.
- **Flujo de navegación:** al crear la cuenta se inicia sesión automáticamente y se navega al HomeMenú; «Ya tengo cuenta» vuelve al Login.
- **Manejo de entrada:** correo con formato válido, contraseña de 6 o más caracteres e igual a su confirmación; filas completas tocables de 56 dp.

View de Recuperar contraseña (RecoverPasswordScreen.kt)

- **Propósito:** enviar el correo de restablecimiento con sendPasswordResetEmail.
- **Componentes UI:** barra superior con volver, texto guía, campo de correo, tarjeta de confirmación y dos botones.

- **Flujo de navegación:** vuelve al Login con la barra superior o el botón secundario.
- **Manejo de entrada:** el envío se habilita solo con un correo válido; el mensaje no revela si la cuenta existe (protección contra enumeración de correos).

View HomeMenú (HomeMenuScreen.kt)

- **Propósito:** dar acceso directo a las cuatro funciones de la app.
- **Componentes UI:** saludo con el nombre del perfil y cuatro tarjetas-botón con ícono de 56 dp, título y descripción.
- **Flujo de navegación:** lleva a Escribir, Hablar, BuscarDispositivo y Perfil.
- **Manejo de entrada:** toda la tarjeta es tocable. Con WindowSizeClass se muestra 1 columna en teléfono vertical y 2 columnas en horizontal o tablet.

View Escribir (EscribirScreen.kt)

- **Propósito:** mostrar en letra grande lo que dice otra persona (voz a texto) o lo que el usuario escribe.
- **Componentes UI:** tarjeta de texto a 30 sp, botón Escuchar/Detener (SpeechRecognizer), campo de texto, botones Ver grande, Borrar y Guardar, y la lista de textos guardados.
- **Flujo de navegación:** HomeMenú → Escribir → pantalla grande; Atrás vuelve al HomeMenú.
- **Manejo de entrada:** pide el permiso de micrófono en tiempo de ejecución, muestra resultados parciales y traduce los errores del reconocedor. Los textos se guardan en users/{uid}/phrases (CRUD).

View Hablar (HablarScreen.kt)

- **Propósito:** que el teléfono diga en voz alta lo que el usuario escribe o elige.
- **Componentes UI:** tarjeta con el estado de la voz, campo y botón Decir, chips de categoría con colores obtenidos por Palette, frases rápidas y la lista Mis frases con decir, favorita, editar y eliminar.
- **Flujo de navegación:** se abre desde el HomeMenú o desde el widget, que además dice la frase al abrir.

- **Manejo de entrada:** TtsService informa si el motor quedó listo, si no hay voz en español o si falta el motor (en ese caso la frase se muestra en pantalla). Cada uso suma 1 con `ServerValue.increment` y la lista se ordena por uso.

View BuscarDispositivo (BuscarDispositivoScreen.kt)

- **Propósito:** registrar dispositivos y la ubicación donde quedaron.
- **Componentes UI:** botón Agregar, tarjetas por dispositivo con su última ubicación y fecha, botones Registrar ubicación y Ver en mapa, diálogo con radio buttons de tipo y el fragment de consejos (ViewGroup clásico).
- **Flujo de navegación:** BuscarDispositivo → app de mapas mediante un intent geo: construido con `toUri()`.
- **Manejo de entrada:** pide los permisos de ubicación en tiempo de ejecución y usa `FusedLocationProviderClient.getCurrentLocation` con `await()`. Los dispositivos se guardan en `users/{uid}/devices` (CRUD).

View Perfil (ProfileScreen.kt)

- **Propósito:** ver y editar los datos del usuario, cerrar sesión o eliminar la cuenta.
- **Componentes UI:** tarjeta de datos, campo de nombre, tres botones grandes y un diálogo de confirmación.
- **Flujo de navegación:** al cerrar sesión o eliminar la cuenta, la guardia de sesión vuelve al Login y limpia la pila.
- **Manejo de entrada:** guardar se habilita solo si el nombre cambió; eliminar exige la contraseña (reautenticación) y borra `users/{uid}` antes de la cuenta.

Adaptabilidad de las views

Las views se revisaron en el emulador en cuatro configuraciones (capturas en el Anexo 3):

- **Teléfono vertical:** menú en una columna y contenido desplazable.
- **Teléfono horizontal:** `WindowSizeClass` detecta la altura reducida y el menú pasa a dos columnas.
- **Pantalla de tablet:** menú en dos columnas y contenido centrado con ancho máximo, para que los textos no se estiren.

- **Letra del sistema en tamaño grande (1,3x):** como todos los textos están en sp, crecen con la configuración del teléfono; se ajustan en varias líneas sin cortarse y la fila de categorías de Hablar se desplaza horizontalmente.

Rendimiento de las pruebas unitarias

Framework	Pruebas	Qué valida	Resultado
JUnit 4	29	Validaciones, mapeo de errores de Firebase, filtros de frases, conversión de datos y orden por uso (TDD)	29/29
Mockito (mockito-kotlin)	22	AuthViewModel, PhraseViewModel y DeviceViewModel con repositorios simulados	22/22
Robolectric	13	Correo con android.util.Patterns, excepciones de Firebase, DAO de Room y ContentProvider	13/13
Espresso / Compose UI test	5	Login → HomeMenú → Hablar, navegación protegida y CRUD de dispositivos en el emulador	5/5
Total	69		69/69 (100 %)

- **TDD:** la función de ordenar frases por uso se escribió primero como prueba fallida (commit test(rojo): 4 de 6 pruebas fallan) y luego se implementó hasta pasar (commit feat(verde): 6 de 6).
- **Cobertura (Kover):** 74,6 % de las líneas de los ViewModels y 18,5 % del total. El total es bajo porque Kover solo mide las pruebas JVM; las pantallas Compose se prueban en el emulador.
- **Firestore Test Lab:** Robo Test correcto en Pixel 5, nivel de API 30. Se realizaron 87 acciones, 4 actividades en 18 pantallas.

Los reportes HTML de pruebas y cobertura se adjuntan en el Anexo 4.

Restricciones del proyecto.

- **Conexión a internet:** la autenticación y la sincronización requieren red. Sin conexión, la app avisa y guarda los cambios localmente hasta reconectar, pero no permite iniciar sesión por primera vez.
- **Servicios de Google:** Firebase y la ubicación dependen de Google Play Services; no funcionan en dispositivos sin ellos.
- **Versión de Android:** minSdk 24 (Android 7.0); dispositivos anteriores no pueden instalar la app.
- **Motores de voz:** texto a voz depende del motor TTS del fabricante (si falta, la frase se muestra en pantalla y se ofrece instalar voces). Voz a texto requiere un servicio de reconocimiento y, por lo general, internet.
- **Permisos:** micrófono y ubicación se piden en tiempo de ejecución; si se niegan, esas funciones quedan deshabilitadas con un mensaje explicativo.
- **Distribución:** Firebase App Distribution solo entrega la app a testers invitados; no reemplaza la publicación en Google Play.
- **Plataforma gratuita:** el proyecto usa el plan gratuito de Firebase (Spark), con límites de almacenamiento y conexiones simultáneas.
- **Entorno de pruebas:** en el emulador la voz a texto no recibe audio y no hay voz es-CL (se usa es-US). La tablet se simuló cambiando la resolución y la densidad del emulador.

Estructura de desglose de trabajo.

Definición de Diagrama y diccionario EDT. Adjuntar diagrama y diccionario.

El trabajo de la Semana 8 se desglosa en cinco paquetes, con 26 elementos de tercer nivel:

- **1.1 Back end Firebase:** configuración, autenticación, repositorios y CRUD, reglas de seguridad y conectividad.
- **1.2 Front end (views):** Login, Registro, Recuperar, HomeMenú, Escribir, Hablar, BuscarDispositivo y Perfil.
- **1.3 Componentes Android y KTX:** Room y ContentProvider, widget, fragment con ViewGroup, Palette y extensiones KTX.
- **1.4 Pruebas:** JUnit y TDD, Mockito, Robolectric, Espresso/Compose UI test, cobertura y Firebase Test Lab.
- **1.5 Firma y distribución:** keystore y firma, APK release, Firebase App Distribution, repositorio Git y ZIP.

El diagrama EDT se presenta en el Anexo 1 y el diccionario EDT en el Anexo 2.

Resumen de riesgos

Listado de riesgos relacionado al desarrollo de S.W. Indicar riesgo, etapa o fase en que se presenta, la probabilidad de que ocurra, magnitud o impacto de este riesgo por etapa en el proceso.

Probabilidad: alta, media, baja.

Impacto: alto, significativo, moderado, inferior y bajo.

Riesgo	Fase	Probabilidad	Impacto	Acción de mitigación
Incompatibilidad de interfaz en pantallas pequeñas	Diseño	Media	Alto	Acción: WindowSizeClass (1 o 2 columnas), contenido desplazable, ancho máximo de 720 dp, textos en sp y controles de 56 dp. Resultado: sin cortes ni desbordes en teléfono vertical y horizontal, en tablet ni con letra del sistema grande (Anexo 3). Mitigado.
Fallo en navegación por rutas mal estructuradas	Desarrollo	Baja	Alto	Acción: rutas como constantes, dos grafos (público y protegido) y una guardia de sesión que reemplaza la pila. Resultado: la prueba NavegacionProtegidaTest pasa; tras cerrar sesión «Atrás» no vuelve al Home. Mitigado.
Ineficiencia en la validación del formulario	Desarrollo	Media	Moderado	Acción: validaciones con android.util.Patterns, botones deshabilitados con datos inválidos, errores de Firebase en español y .validate en las reglas. Resultado: pruebas JUnit y Robolectric en verde. Mitigado.
Pérdida de código o cambios desincronizados	Gestión	Baja	Alto	Acción: commits pequeños y descriptivos, normalización de fines de línea, etiqueta v1.0.0 y push al repositorio remoto en

				GitHub. Resultado: historial completo publicado. Mitigado.
Bloqueo de la app por falta de conectividad	Pruebas	Media	Moderado	Acción: observador de conectividad con aviso visible, persistencia offline de Realtime Database y escrituras con espera acotada. Resultado: en modo avión la frase se guardó localmente y se sincronizó al volver la red. Mitigado.
Exposición de datos por base de datos en modo prueba (nuevo)	Back end	Alta	Alto	Acción: reglas definitivas (acceso solo a users/\$uid de su dueño y validación de campos) y verificación de que el código no escribe fuera de users/{uid}. Resultado: reglas publicadas antes de la entrega. Mitigado.
Pérdida o filtración de la clave de firma (nuevo)	Distribución	Baja	Alto	Acción: keystore fuera del repositorio, contraseña aleatoria en keystore.properties ignorado por Git. Resultado: ningún secreto versionado. Pendiente: guardar un respaldo cifrado del keystore.

Seguimiento de los riesgos identificados en la Sumativa 2

La Sumativa 2 identificó cinco riesgos adicionales. Su estado en esta entrega es el siguiente:

- **Almacenamiento de contraseñas sin cifrar:** la app ya no guarda contraseñas. Firebase Authentication las administra en sus servidores (almacenadas con hash) y en el teléfono solo queda la sesión. Estado: resuelto.
- **Volatilidad de datos por persistencia en memoria:** los datos se guardan en Realtime Database con persistencia local, y las frases además en una caché Room. Se conservan al cerrar la app. Estado: resuelto.

- **Inicialización asíncrona del motor de voz (TTS):** TtsService expone el resultado de onInit como estado (preparando, lista, sin español, sin motor) y la view Hablar lo muestra; no se intenta hablar antes de que el motor esté listo. Estado: resuelto.
- **Falta de manejo de estado sin conexión:** ConnectivityObserver muestra el aviso «Sin conexión a internet» y Firebase guarda los cambios localmente hasta reconectar. Estado: resuelto.
- **Fragmentación del motor TTS entre fabricantes:** se buscan voces en es-CL, es-419, es-US y es-ES; si no hay motor, la frase se muestra en pantalla y se ofrece instalar voces. En el emulador, sin voz es-CL, se usó es-US correctamente. Estado: resuelto.

Definición de artefactos.

Listar y describir los artefactos o entregables que serán administrados y entregados durante el desarrollo del proyecto.

Artefacto	Descripción
Código fuente versionado	Proyecto Android Studio en Kotlin, organizado en capas (ui, data, domain, provider, widget), con su historial en GitHub y la etiqueta v1.0.0.
Repositorio Git	https://github.com/rodolfoulloa/DSY2204_Exp1_S2_accesip lus , con commits incrementales, incluido el ciclo TDD.
APK release firmado	accesiplus-1.0.0.apk (versionCode 2), firmado con keystore propio (esquema APK Signature v2) y reducido con R8.
Publicación del APK	Versión 1.0.0 (2) distribuida a testers en Firebase App Distribution.
Back end Firebase	Proyecto accesiplus con Authentication y Realtime Database; reglas en database.rules.json publicadas en la consola.
Suite de pruebas y reportes	69 pruebas (JUnit, Mockito, Robolectric y Compose UI test), reportes HTML y cobertura Kover.
Informe técnico (PDF)	Este documento: alcance, views, pruebas, restricciones, EDT, riesgos y condiciones de aceptación.
Paquete de entrega (.ZIP)	ZIP generado desde GitHub con el código del proyecto, más este informe en PDF, para subir al AVA.

Condiciones de aceptación para cierre del proyecto

Condiciones que se deben cumplir para dar termino al proyecto y margen de tolerancia de aceptación de defectos.

- **Criterio de autenticación:** el 100 % de los inicios de sesión correctos lleva al HomeMenú y «Atrás» no regresa al Login. **Resultado:** Cumplido (LoginHablarTest y NavegacionProtegidaTest).
- **Criterio de protección:** sin sesión no se puede abrir ninguna view interna, y tras cerrar sesión no se vuelve a ellas. **Resultado:** Cumplido (NavegacionProtegidaTest).
- **Criterio de validación:** ningún formulario se envía con correo inválido, contraseña corta o contraseñas distintas. **Resultado:** Cumplido (ValidacionesTest, ValidacionCorreoRobolectricTest y AuthViewModelTest).
- **Criterio de persistencia:** las frases, los dispositivos y el perfil se conservan al cerrar la app y solo los ve su dueño. **Resultado:** Cumplido (Realtime Database con reglas por usuario).
- **Criterio de robustez:** el texto a voz no provoca cierres forzados; si no hay motor, la frase se muestra en pantalla. **Resultado:** Cumplido (PhraseViewModelTest y TtsService).
- **Criterio de distribución:** el APK está firmado con keystore propio y publicado en una plataforma gratuita. **Resultado:** Cumplido (apksigner verify y Firebase App Distribution).
- **Criterio de adaptabilidad:** las views se ven completas en teléfono vertical y horizontal, en tablet y con letra del sistema grande. **Resultado:** Cumplido (capturas del Anexo 3).
- **Margen de tolerancia de defectos:** 0 % para errores críticos o altos (cierres forzados, fallos de navegación o de datos) y hasta 5 % para defectos cosméticos que no afecten la usabilidad. **Resultado:** 69 de 69 pruebas aprobadas y ningún cierre forzado en el recorrido de las views.

Anexos

Anexo 1: Diagrama EDT

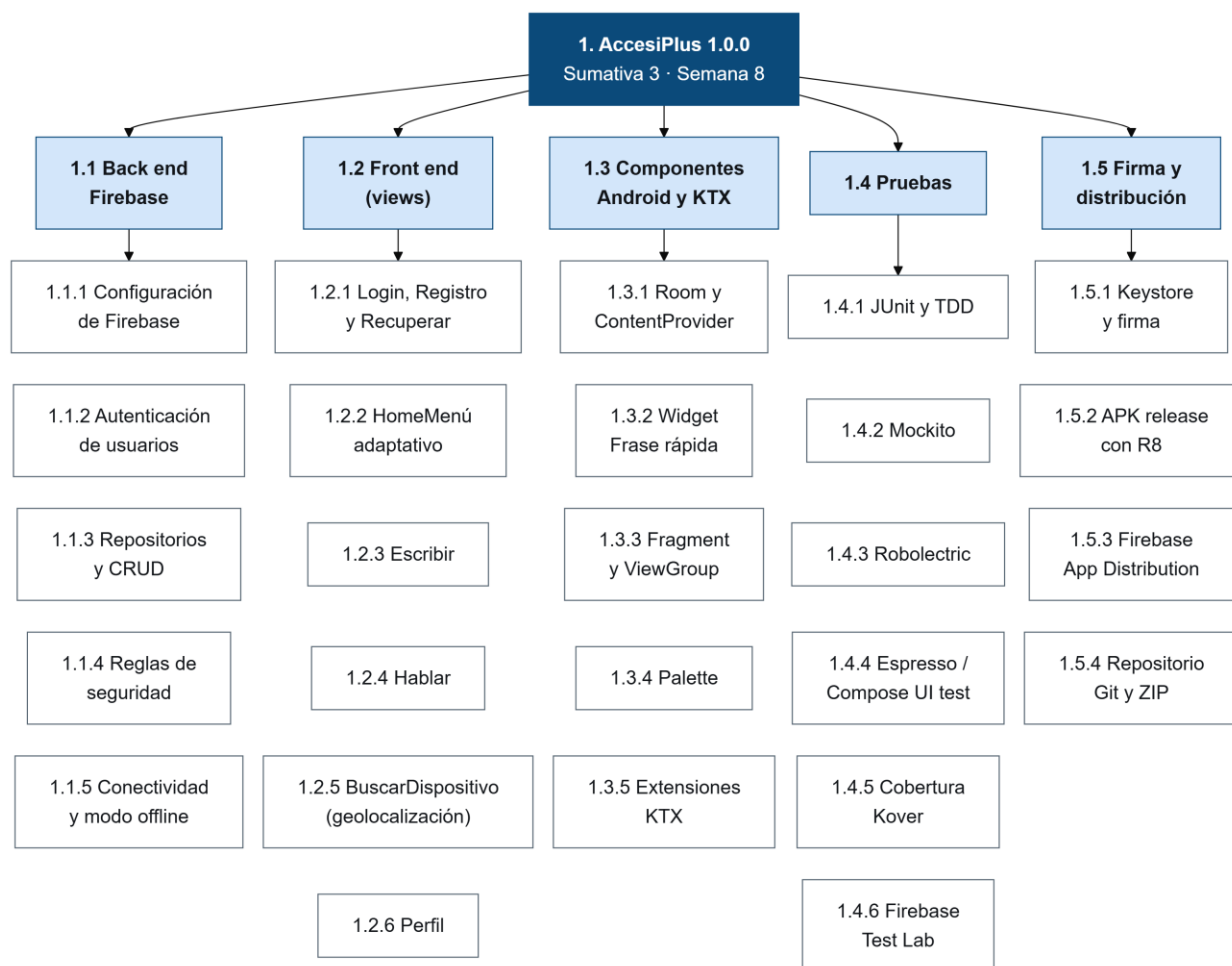


Figura 1. Diagrama EDT de AccesiPlus 1.0.0 (Semana 8).

Anexo 2. Diccionario EDT

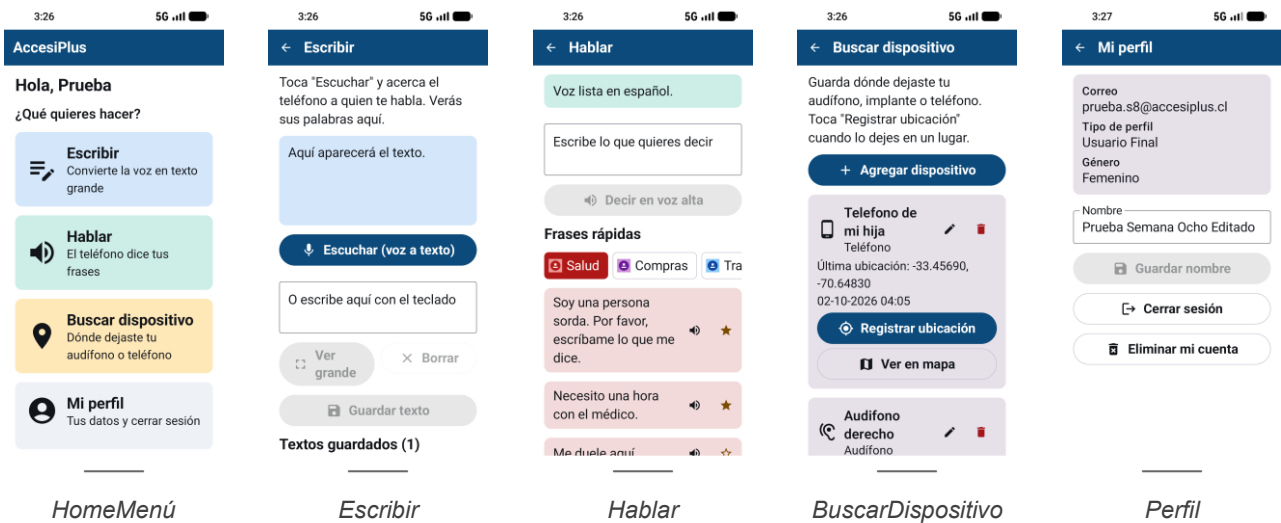
Código	Elemento EDT	Descripción técnica del trabajo realizado	Entregable
1.1.1	Configuración de Firebase	Registro de la app en el proyecto accesiplus, plugin google-services, Firebase BoM 34.19.0 y persistencia offline.	google-services.json, AccesiPlusApp.kt

1.1.2	Autenticación de usuarios	Login, registro, recuperación, reautenticación y eliminación con Firebase Authentication usando <code>await()</code> y <code>callbackFlow</code> .	<code>AuthRepository.kt</code>
1.1.3	Repositorios y CRUD	CRUD de perfil, frases y dispositivos en <code>users/{uid}</code> ; todas las rutas pasan por <code>RutasFirebase</code> .	<code>UserRepository</code> , <code>PhraseRepository</code> , <code>DeviceRepository</code>
1.1.4	Reglas de seguridad	Reglas por usuario (<code>auth.uid === \$uid</code>) con validación de campos, tipos, largos y rangos.	<code>database.rules.json</code>
1.1.5	Conectividad y modo offline	Observador de red con <code>callbackFlow</code> , aviso sin conexión y escrituras en cola.	<code>ConnectivityObserver.kt</code>
1.2.1	Login, Registro y Recuperar	Views de autenticación conectadas a Firebase, con validaciones y errores en español.	<code>LoginScreen</code> , <code>RegisterScreen</code> , <code>RecoverPasswordScreen</code>
1.2.2	HomeMenú adaptativo	Menú de cuatro tarjetas con <code>WindowSizeClass</code> (1 o 2 columnas).	<code>HomeMenuScreen.kt</code>
1.2.3	Escribir	Voz a texto con <code>SpeechRecognizer</code> , texto grande y CRUD de textos.	<code>EscribirScreen.kt</code> , <code>ReconocedorVoz.kt</code>
1.2.4	Hablar	Texto a voz con <code>TtsService</code> , frases rápidas por categoría y CRUD de frases propias.	<code>HablarScreen.kt</code> , <code>TtsService.kt</code>
1.2.5	BuscarDispositivo	CRUD de dispositivos, ubicación con <code>FusedLocationProviderClient</code> y apertura en mapa.	<code>BuscarDispositivoScreen.kt</code>
1.2.6	Perfil	Edición de nombre, cierre de sesión y eliminación de la cuenta.	<code>ProfileScreen.kt</code>
1.3.1	Room y ContentProvider	Caché local de frases sincronizada desde Firebase y expuesta con un <code>ContentProvider</code> de solo lectura.	<code>BaseLocal.kt</code> , <code>PhrasesProvider.kt</code>
1.3.2	Widget Frase rápida	App Widget que muestra la última favorita y abre Hablar para decirla.	<code>FraseRapidaWidget.kt</code>
1.3.3	Fragment y ViewGroup	Fragment de consejos con <code>LinearLayout</code> clásico integrado en <code>Compose</code> con <code>AndroidFragment</code> .	<code>ConsejosFragment.kt</code>
1.3.4	Palette	Colores de categoría extraídos de imágenes con contraste mínimo 4.5:1.	<code>ColoresCategoria.kt</code>
1.3.5	Extensiones KTX	Uso de <code>core-ktx</code> , <code>lifecycle</code> , <code>Firebase</code> , <code>coroutines</code> , <code>room-ktx</code> , <code>palette-ktx</code> y <code>fragment-ktx</code> .	Código fuente
1.4.1	JUnit y TDD	Pruebas de lógica pura y ciclo rojo-verde de <code>OrdenFrases</code> .	29 pruebas
1.4.2	Mockito	Pruebas de <code>ViewModels</code> con repositorios simulados.	22 pruebas
1.4.3	Robolectric	Pruebas con APIs de Android en la JVM: <code>Patterns</code> , <code>Room</code> y <code>ContentProvider</code> .	13 pruebas
1.4.4	Espresso / Compose UI test	Pruebas instrumentadas de punta a punta en el emulador.	5 pruebas
1.4.5	Cobertura Kover	Medición de cobertura de las pruebas JVM.	Reporte HTML y XML
1.4.6	Firebase Test Lab	Robo test en la infraestructura de Firebase Test Lab (Pixel 5, API 30).	Resultado de Test Lab
1.5.1	Keystore y firma	Keystore propio PKCS12 fuera del repositorio y <code>signingConfig</code> en Gradle.	<code>accesiplus.jks</code> (privado)

1.5.2	APK release con R8	Generación del APK firmado, optimizado y verificado con apksigner.	accesiplus-1.0.0.apk
1.5.3	Firebase App Distribution	Publicación de la versión 1.0.0 (2) a testers.	Versión publicada
1.5.4	Repositorio Git y ZIP	Commits, etiqueta v1.0.0, push a GitHub y ZIP de entrega.	Repositorio y ZIP

Anexo 3. Pruebas de adaptabilidad

Letra del sistema en tamaño grande (1,3×), teléfono Pixel 5 en vertical:



Teléfono en horizontal y pantalla de tablet (menú en dos columnas):

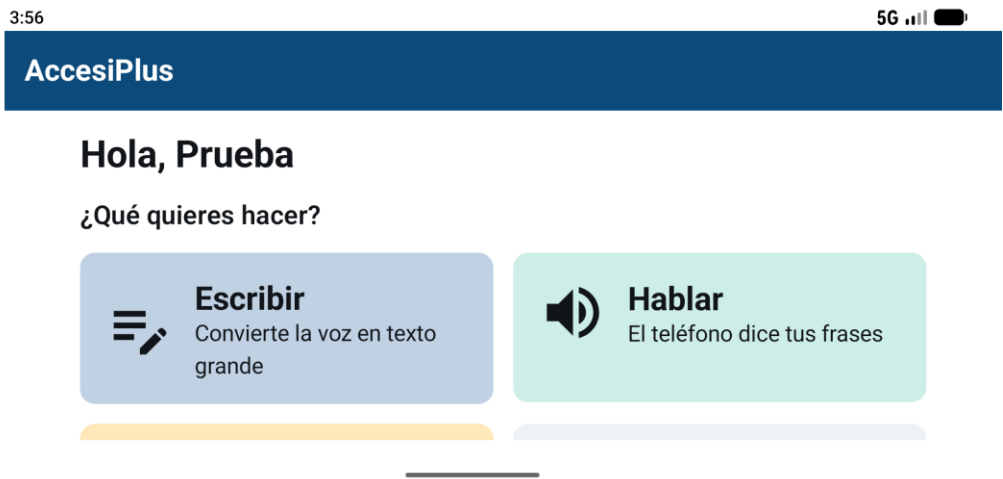


Figura 2. HomeMenú con el teléfono en horizontal.

4:13

5G  

AccesiPlus

Hola, Prueba

¿Qué quieres hacer?



Escribir

Convierte la voz en texto grande



Hablar

El teléfono dice tus frases



Buscar dispositivo

Dónde dejaste tu audífono o teléfono



Mi perfil

Tus datos y cerrar sesión

Figura 3. HomeMenú en pantalla de tablet.

Anexo 4. Reportes de pruebas y cobertura

All Results

● Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

64 tests	0 failures	0 skipped	19.045s duration	100% successful
--------------------	----------------------	---------------------	----------------------------	---------------------------

Child	Name	Tests	Failures	Skipped	Duration	Success rate
MapeoTest	cl.duoc.rulloo.accesiplus.data.model.MapeoTest	5	0	0	0.240s	100%
ErroresFirebaseTest	cl.duoc.rulloo.accesiplus.domain.ErroresFirebaseTest	6	0	0	0.093s	100%
FiltrosFrasesTest	cl.duoc.rulloo.accesiplus.domain.FiltrosFrasesTest	8	0	0	0.022s	100%
OrdenFrasesTest	cl.duoc.rulloo.accesiplus.domain.OrdenFrasesTest	6	0	0	0.019s	100%
ValidacionesTest	cl.duoc.rulloo.accesiplus.domain.ValidacionesTest	4	0	0	0.070s	100%
FraseDaoRobolectricTest	cl.duoc.rulloo.accesiplus.robolectric.FraseDaoRobolectricTest	5	0	0	8.344s	100%
PhrasesProviderRobolectricTest	cl.duoc.rulloo.accesiplus.robolectric.PhrasesProviderRobolectricTest	5	0	0	0.280s	100%
ValidacionCorreoRobolectricTest	cl.duoc.rulloo.accesiplus.robolectric.ValidacionCorreoRobolectricTest	3	0	0	0.129s	100%
AuthViewModelTest	cl.duoc.rulloo.accesiplus.ui.auth.AuthViewModelTest	7	0	0	4.111s	100%
DeviceViewModelTest	cl.duoc.rulloo.accesiplus.ui.devices.DeviceViewModelTest	7	0	0	0.450s	100%
PhraseViewModelTest	cl.duoc.rulloo.accesiplus.ui.phrases.PhraseViewModelTest	8	0	0	0.451s	100%

Generated by Gradle 9.5.0 at 02-10-2026, 1:43:04 a. m.

Figura 4. Reporte de pruebas unitarias: 64 pruebas, 100 % de éxito.

Test Summary

5 tests	0 failures	0 skipped	46.477s duration	100% successful
-------------------	----------------------	---------------------	----------------------------	---------------------------

Packages

Classes

Package	Tests	Failures	Skipped	Duration	Success rate
cl.duoc.rulloo.accesiplus	5	0	0	46.477s	100%

Generated by Gradle 9.5.0 at 02-10-2026, 1:42:19 a. m.

Figura 5. Reporte de pruebas instrumentadas en el emulador: 5 pruebas.

Current scope: app | all classes

app: Overall Coverage Summary

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
all classes	31,8% (42/132)	16,7% (92/550)	19,4% (176/909)	18,5% (310/1680)	11,9% (2809/23632)

Coverage Breakdown

Package	Class, %	Method, %	Branch, %	Line, %	Instruction, %
cl.duoc.rulloo.accesiplus	0% (0/2)	0% (0/14)	0% (0/10)	0% (0/37)	0% (0/360)
cl.duoc.rulloo.accesiplus.data.connectivity	0% (0/3)	0% (0/8)	0% (0/8)	0% (0/17)	0% (0/114)
cl.duoc.rulloo.accesiplus.data.local	45,5% (5/11)	38,9% (7/18)	16,7% (1/6)	51,4% (19/37)	42,1% (136/323)
cl.duoc.rulloo.accesiplus.data.location	33,3% (1/3)	25% (1/4)	0% (0/4)	11,1% (1/9)	11% (12/109)
cl.duoc.rulloo.accesiplus.data.model	57,1% (4/7)	64,7% (11/17)	73,3% (63/86)	59,7% (71/119)	70,6% (612/867)
cl.duoc.rulloo.accesiplus.data.remote	0% (0/6)	0% (0/12)	0% (0/4)	0% (0/16)	0% (0/158)
cl.duoc.rulloo.accesiplus.data.repository	0% (0/22)	0% (0/55)	0% (0/28)	0% (0/69)	0% (0/1389)
cl.duoc.rulloo.accesiplus.data.tts	33,3% (1/3)	16,7% (1/6)	0% (0/18)	13,8% (4/29)	15,5% (30/193)
cl.duoc.rulloo.accesiplus.di	0% (0/1)	0% (0/3)		0% (0/15)	0% (0/202)
cl.duoc.rulloo.accesiplus.domain	100% (4/4)	83,3% (15/18)	91,3% (42/46)	97,4% (38/39)	95,5% (299/313)
cl.duoc.rulloo.accesiplus.provider	100% (2/2)	90% (9/10)	62,5% (5/8)	91,3% (21/23)	75% (87/116)
cl.duoc.rulloo.accesiplus.ui	0% (0/2)	0% (0/7)		0% (0/15)	0% (0/87)
cl.duoc.rulloo.accesiplus.ui.auth	75% (9/12)	19,5% (16/82)	17,6% (18/102)	17,7% (44/249)	11,9% (473/3984)
cl.duoc.rulloo.accesiplus.ui.components	0% (0/2)	0% (0/16)	0% (0/56)	0% (0/89)	0% (0/1547)
cl.duoc.rulloo.accesiplus.ui.devices	66,7% (6/9)	20,3% (13/64)	20,1% (27/134)	27,7% (61/220)	18,8% (606/3222)
cl.duoc.rulloo.accesiplus.ui.escribir	0% (0/4)	0% (0/47)	0% (0/67)	0% (0/130)	0% (0/1885)
cl.duoc.rulloo.accesiplus.ui.hablar	20% (1/5)	0% (0/42)	0% (0/104)	0% (0/150)	0,1% (4/2676)
cl.duoc.rulloo.accesiplus.ui.home	0% (0/7)	0% (0/14)	0% (0/36)	0% (0/59)	0% (0/820)
cl.duoc.rulloo.accesiplus.ui.navigation	0% (0/7)	0% (0/34)	0% (0/48)	0% (0/82)	0% (0/1399)
cl.duoc.rulloo.accesiplus.ui.phrases	80% (8/10)	43,6% (17/39)	24,4% (20/82)	35% (49/140)	27% (539/1993)
cl.duoc.rulloo.accesiplus.ui.profile	0% (0/7)	0% (0/32)	0% (0/52)	0% (0/104)	0% (0/1647)
cl.duoc.rulloo.accesiplus.widget	33,3% (1/3)	25% (2/8)	0% (0/10)	6,2% (2/32)	4,8% (11/228)

generated on 2025-10-02 01:43

Figura 6. Reporte de cobertura Kover.

Anexo 5. Firma y publicación del APK

Dato	Valor
Keystore	PKCS12, RSA 2048, alias accesiplus, validez de 10.000 días
Certificado	CN=Rodolfo Ulloa, OU=DSY2204, O=Duoc UC, L=Santiago, C=CL
SHA-256 del certificado	2f2a411c732e30a327eea309c44166de473cccc8b19da85e197f79a54140b4b6
Verificación (apksigner)	Verifies — APK Signature Scheme v2
APK	accesiplus-1.0.0.apk, 2.475.558 bytes, versionCode 2, versionName 1.0.0
SHA-256 del APK	f61ada47267d0ce95539290fb2b97b6afc2bf04f52cafc1f3572d65b8ace6e64

1.0.0 (2)

2 de octubre de 2026 a las 11:32:41 p.m. UTC-3

AccesiPlus 1.0.0 – Firebase Auth y Realtime Database...

Invitado

2

Aceptada

1

Descargado

1

Comentarios

0

Copiar

Descargar

Borrar

Ejecutar pruebas

Verificadores

Notas de la versión

Comentarios de verificadores

Agente de pruebas de apps PREVIEW

Agregar verificadores o grupos

 Rodolfo Ulloa rodolfou@gmail.com	✓ — ✓ — ✓ Descargado
 enr.urra@profesor.duoc.cl	✓ Invitado

Figura 7. Versión 1.0.0 (2) publicada en Firebase App Distribution, con sus testers.

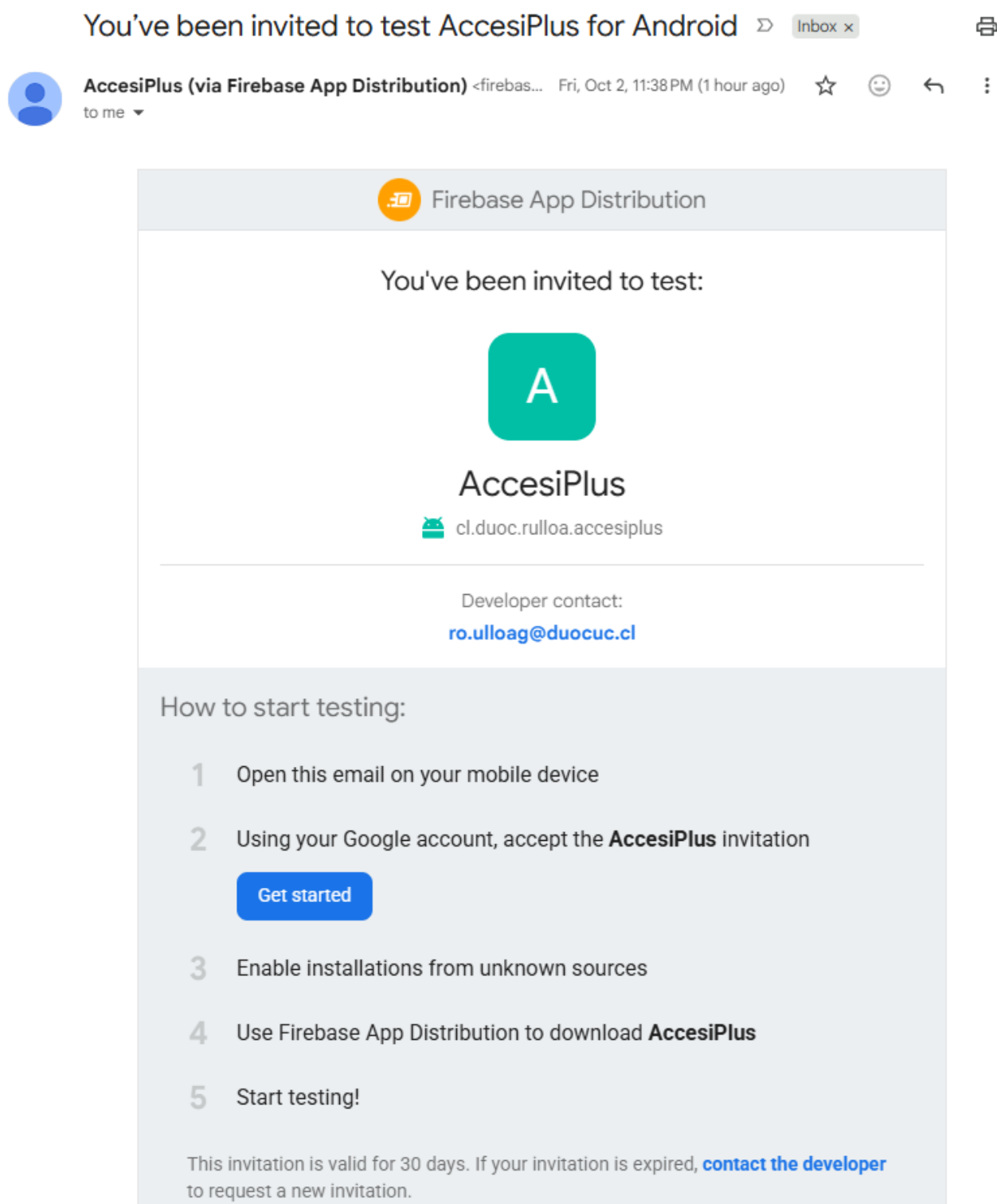
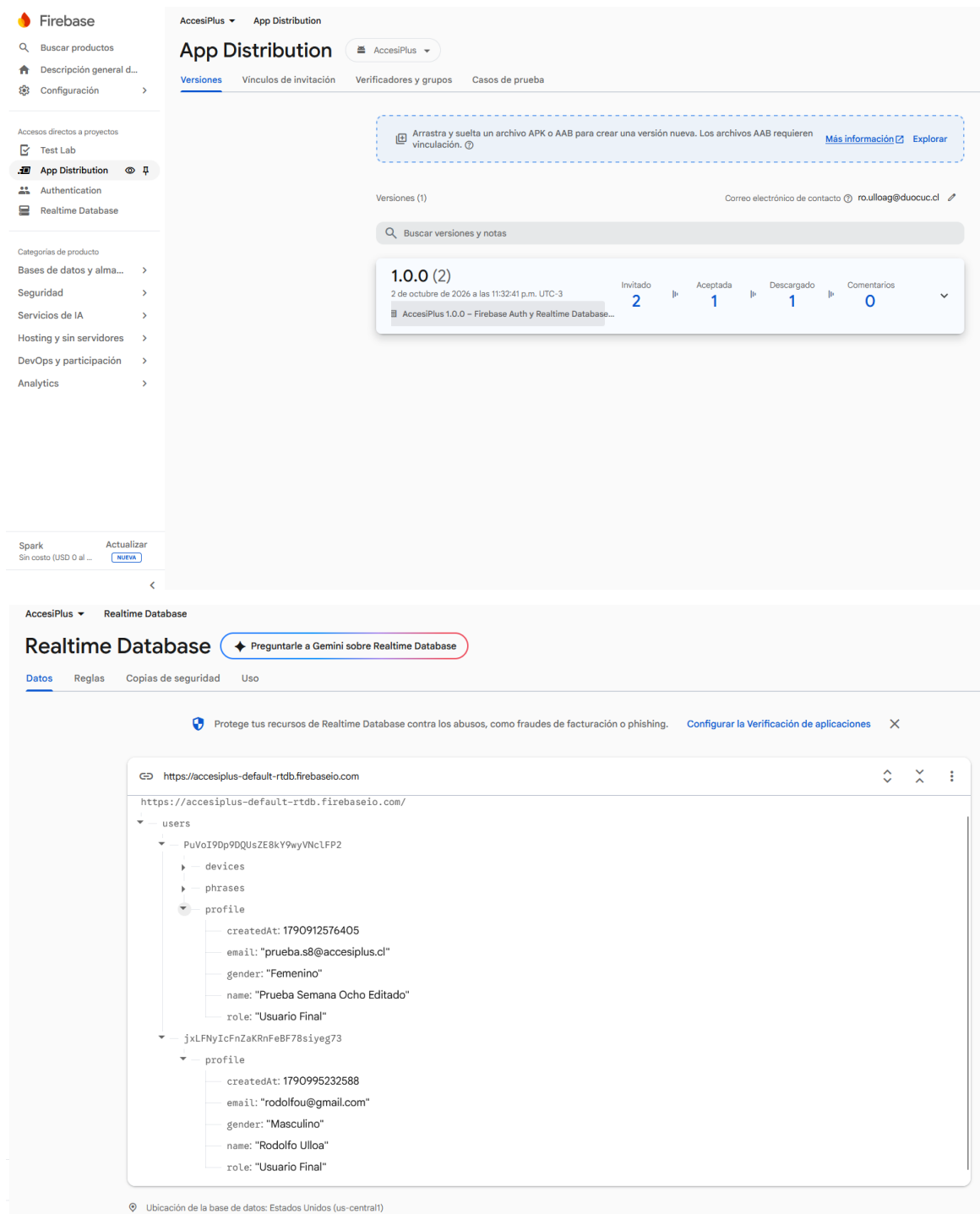


Figura 8. Correo de invitación a testers enviado por Firebase App Distribution.

Anexo 6. Back end en la consola de Firebase



The image shows two screenshots of the Firebase console interface.

Top Screenshot: App Distribution

- Header:** AccesiPlus, App Distribution
- Navigation:** Versiones, Vínculos de invitación, Verificadores y grupos, Casos de prueba
- Instructions:** Arrastra y suelta un archivo APK o AAB para crear una versión nueva. Los archivos AAB requieren vinculación.
- Versión 1.0.0 (2):**
 - Fecha: 2 de octubre de 2026 a las 11:32:41 p.m. UTC-3
 - Invitado: 2
 - Aceptada: 1
 - Descargado: 1
 - Comentarios: 0
 - Detalle: AccesiPlus 1.0.0 - Firebase Auth y Realtime Database...

Bottom Screenshot: Realtime Database

- Header:** AccesiPlus, Realtime Database
- Navigation:** Datos, Reglas, Copias de seguridad, Uso
- Alert:** Protege tus recursos de Realtime Database contra los abusos, como fraudes de facturación o phishing.
- URL:** https://accesiplus-default-rtadb.firebaseio.com/
- Tree Structure:**
 - users
 - PuVoI9Dp9DQUsZE8kY9wyVNcLFP2
 - devices
 - phrases
 - profile
 - createdAt: 1790912576405
 - email: "prueba.s8@accesiplus.cl"
 - gender: "Femenino"
 - name: "Prueba Semana Ocho Editado"
 - role: "Usuario Final"
 - jxLFNyIcFnZaKrnFeBF78siyeg73
 - profile
 - createdAt: 1790995232588
 - email: "rodolfou@gmail.com"
 - gender: "Masculino"
 - name: "Rodolfo Ulloa"
 - role: "Usuario Final"

Figura 9. Consola de Firebase: App Distribution y datos de users/{uid} en Realtime Database.

AccesiPlus ▾

Realtime Database

◆ Preguntarle a Gemini sobre Realtime Database

Datos

Reglas

Copias de seguridad

Uso

Zona de pruebas de reglas

```

1  {
2  v {
3    "rules": {
4      ".read": false,
5      ".write": false,
6      "users": {
7        "$uid": {
8          ".read": "auth != null && auth.uid === $uid",
9          ".write": "auth != null && auth.uid === $uid",
10         "profile": {
11           ".validate": "newData.hasChildren(['name', 'email', 'createdAt'])",
12           "name": { ".validate": "newData.isString() && newData.val().length > 0 && newData.val().length <= 80" },
13           "email": { ".validate": "newData.isString() && newData.val() === auth.token.email" },
14           "role": { ".validate": "newData.isString() && newData.val().length <= 40" },
15           "gender": { ".validate": "newData.isString() && newData.val().length <= 20" },
16           "createdAt": { ".validate": "newData.isNumber()" },
17           "$otro": { ".validate": false }
18         },
19         "phrases": {
20           "$phraseId": {
21             ".validate": "newData.hasChildren(['text', 'category', 'createdAt'])",
22             "text": { ".validate": "newData.isString() && newData.val().length > 0 && newData.val().length <= 500" },
23             "category": { ".validate": "newData.isString() && newData.val().length <= 30" },
24             "favorite": { ".validate": "newData.isBoolean()" },
25             "uses": { ".validate": "newData.isNumber() && newData.val() >= 0" },
26             "createdAt": { ".validate": "newData.isNumber()" },
27             "updatedAt": { ".validate": "newData.isNumber()" },
28             "$otro": { ".validate": false }
29           },
30         },
31         "devices": {
32           "$deviceId": {
33             ".validate": "newData.hasChildren(['name', 'type', 'createdAt'])",
34             "name": { ".validate": "newData.isString() && newData.val().length > 0 && newData.val().length <= 60" },
35             "type": { ".validate": "newData.isString() && newData.val().length <= 30" },
36             "notes": { ".validate": "newData.isString() && newData.val().length <= 300" },
37             "lat": { ".validate": "newData.isNumber() && newData.val() >= -90 && newData.val() <= 90" },
38             "lon": { ".validate": "newData.isNumber() && newData.val() >= -180 && newData.val() <= 180" },
39             "locationAt": { ".validate": "newData.isNumber()" },

```

Figura 10. Reglas de seguridad publicadas en Realtime Database.

AccesiPlus ▾

Test Lab > AccesiPlus > matrix-1sgtkOn333fdn >

Prueba Robo, Pixel 5, nivel de API 30

⚠ **Advertencia:** Firebase Test Lab dejó de estar disponible y se cerrará por completo el 30 de septiembre de 2027. Para mantener tus flujos de trabajo de pruebas, puedes migrar a la [Developer Device Platform](#) en Google Cloud. Consulta la [guía de migración](#) y las [preguntas frecuentes](#) de Developer Device Platform para obtener más detalles.



Figura 11. Resultado del Robo test en Firebase Test Lab (Pixel 5, API 30).



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.